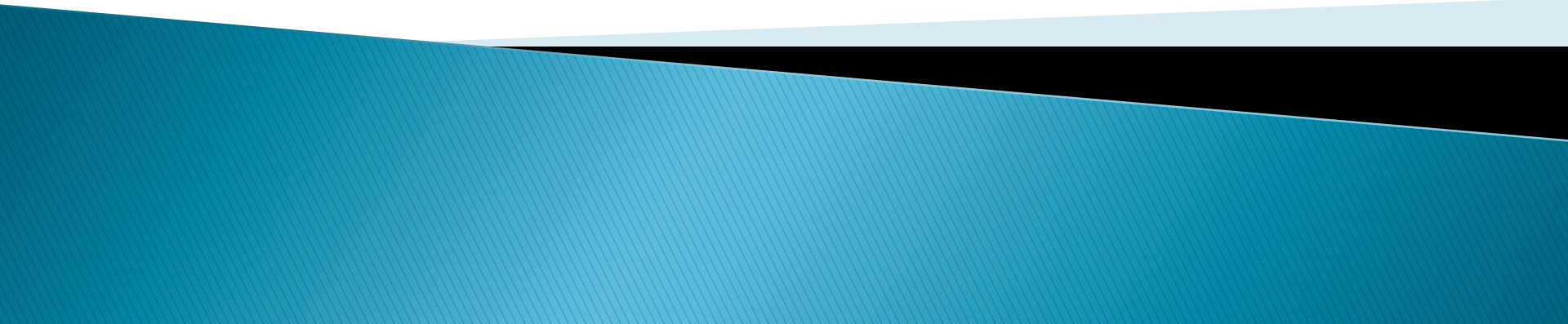
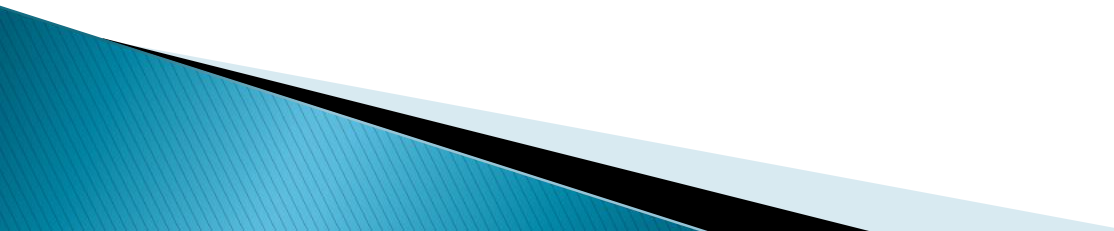


Динамическое программирование



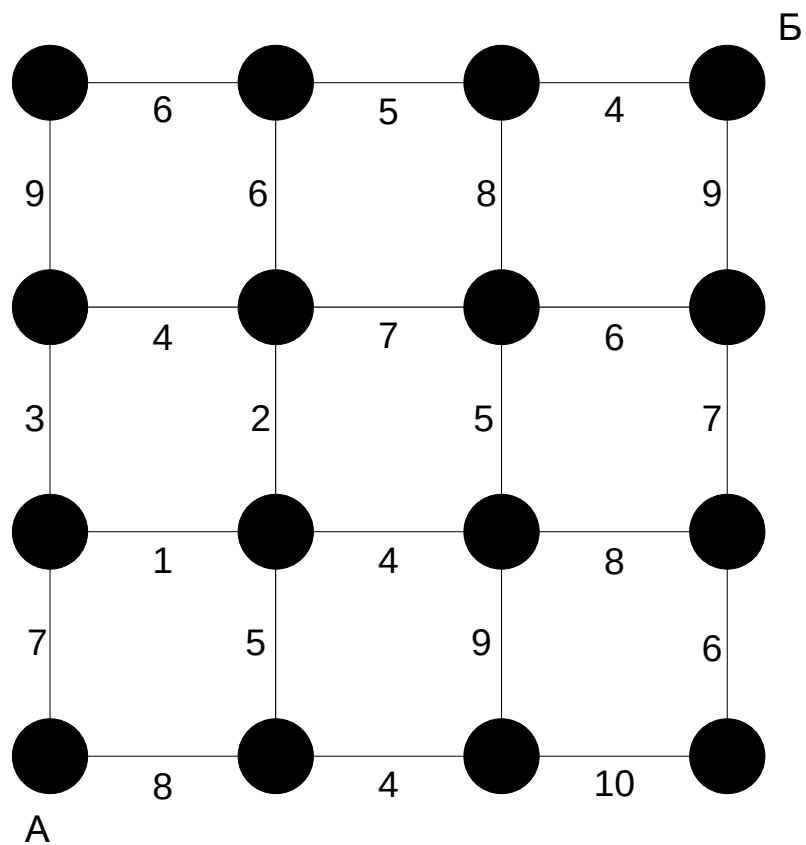
Динамическое программирование

- ▶ Американский математик Ричард Беллман
 - ▶ Суть метода: решение задачи может быть построено из решения подзадач
 - ▶ Аддитивность задачи
 - ▶ Подзадачи перекрывающиеся
 - ▶ Подзадачи не решаются вновь и вновь, а решается один раз и результат хранится
 - ▶ Взаимосвязь подзадач описывается, как правило, рекуррентными соотношениями
- 

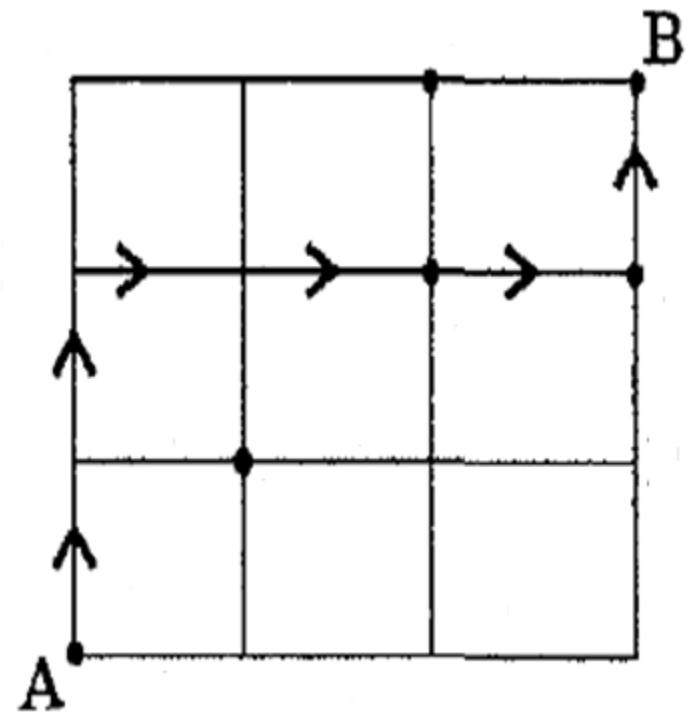
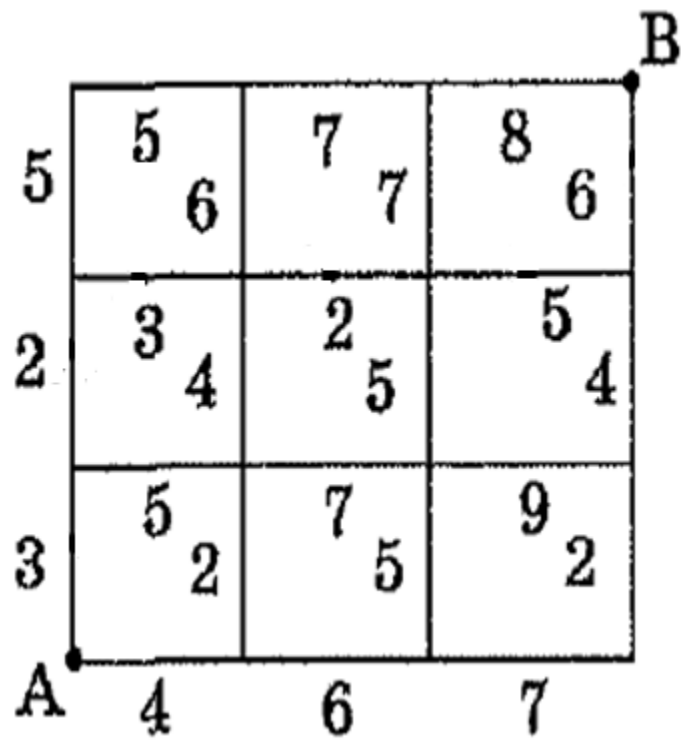
Пример

- ▶ Черепашка перемещается по ребрам клеточного поля из точки А в точку В. За один ход разрешается пройти по одному ребру вправо или вверх. Ребра имеют вес – стоимость перемещения. Найти путь Черепашки с максимальной суммарной стоимостью

Пример



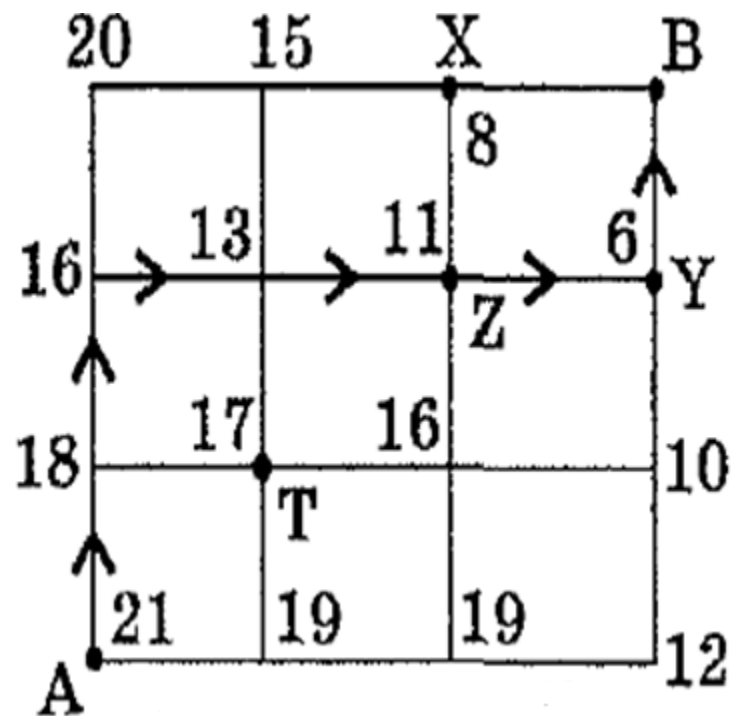
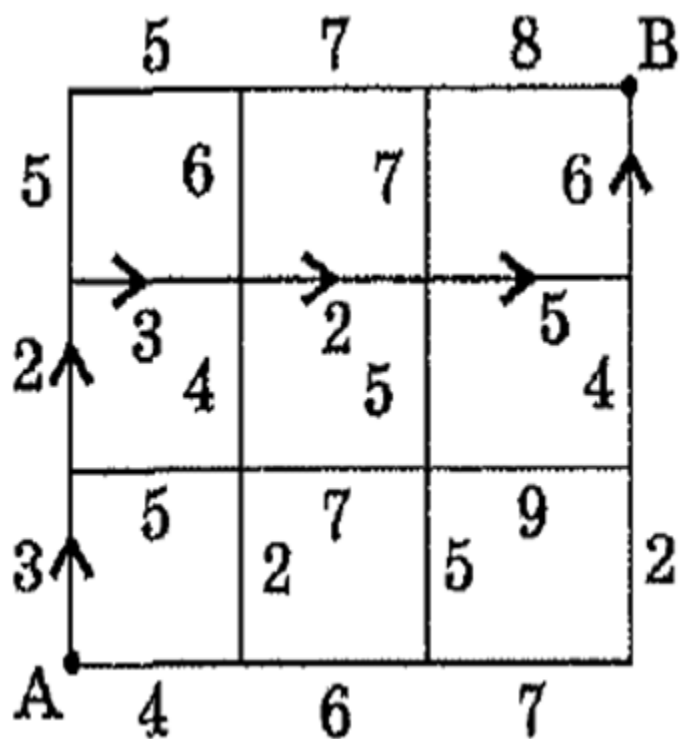
				B
5	5	7	8	
	6	7	6	
2	3	2	5	
	4	5	4	
3	5	7	9	
	2	5	2	
A	4	6	7	



- ▶ Количество возможных путей Черепашки 20 (C_6^3). Мы можем перебрать все пути и выбрать самый быстрый. Это метод полного перебора (ответ – 21)
- ▶ Для вычисления каждого времени требуется пять операций сложения, а для нахождения ответа – 100 операций сложения и 19 операций сравнения. Задача решается вручную

- ▶ Однако при $N=8$ у Черепашки уже 12 870 путей. Подсчет времени для каждого пути требует 15 операций сложения, а в целом — 193 050 сложений и 12 869 сравнений, то есть порядка 200 000 операций
- ▶ Компьютер при быстродействии 1 млн. оп./сек. найдет решение за 0.2 сек.

- ▶ Если $N=30$, количество различных путей $60! \cdot (30! \cdot 30!)$. Порядок этого числа 10^{17} . Количество операций, необходимых для поиска решения, равно $60 \cdot 10^{17}$
- ▶ Компьютер с быстродействием 1 млн. оп./сек. выполнит за год порядка $3.2 \cdot 10^{13}$ операций
- ▶ Сколько месяцев требуется для решения задачи???



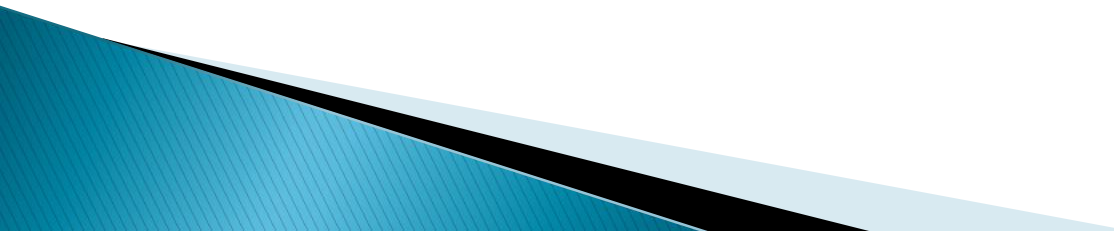
- ▶ Каждому углу присвоим вес, равный минимальному времени движения Черепашки от этого угла до пункта *B*

- ▶ Для каждого угла необходимо выполнить не более двух операций сложения и одной операции сравнения, т.е. три операции
- ▶ При $N=300$ количество операций $3 \cdot 301 \cdot 301 < 1\,000\,000$, и компьютеру потребуется менее одной секунды
- ▶ Много лет при $N=30$ (при полном переборе) и 1 секунда при $N=300$ (динамическое программирование)

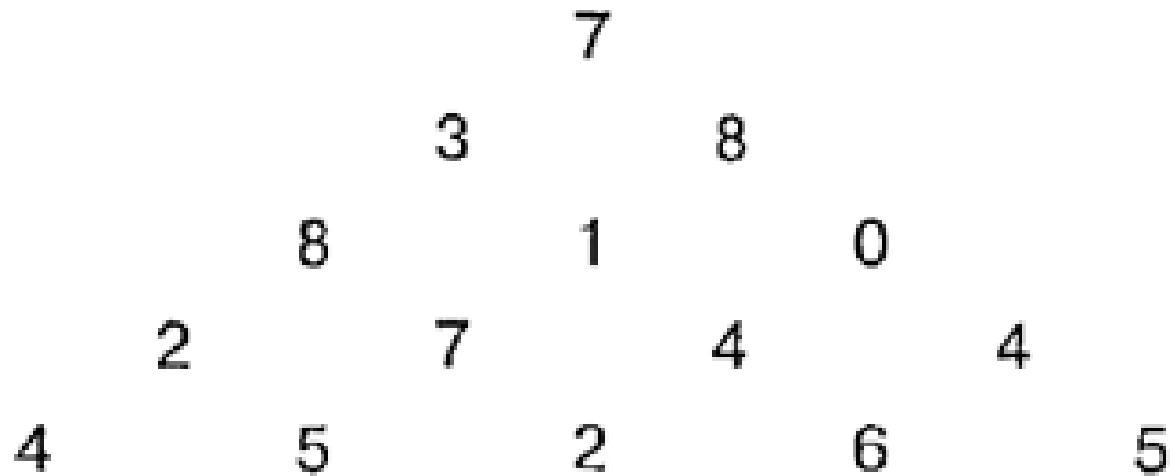
Метод динамического программирования

- ▶ Особенность задачи:
 - для каждого угла найденное число является решением задачи меньшей размерности
 - для каждого угла найденное значение времени не изменяется и может быть использовано на следующих этапах

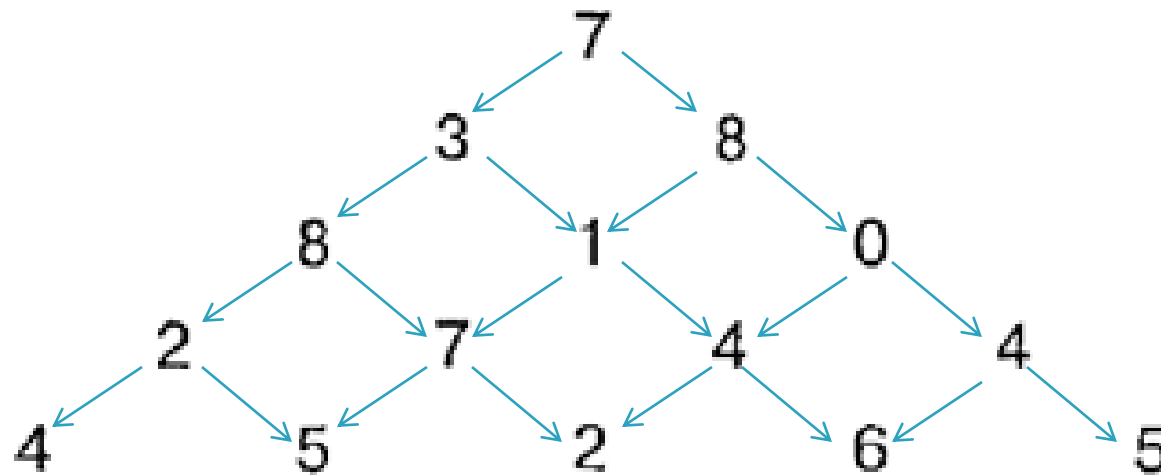
Пример. Треугольник

- ▶ Натуральные числа выстроены в треугольник
 - ▶ Разработать программу для поиска наибольшей возможной суммы при спуске от вершины треугольника к его основанию
 - ▶ Разрешенные перемещения: вниз влево, вниз вправо
 - ▶ Количество строк (n) не более 100
- 

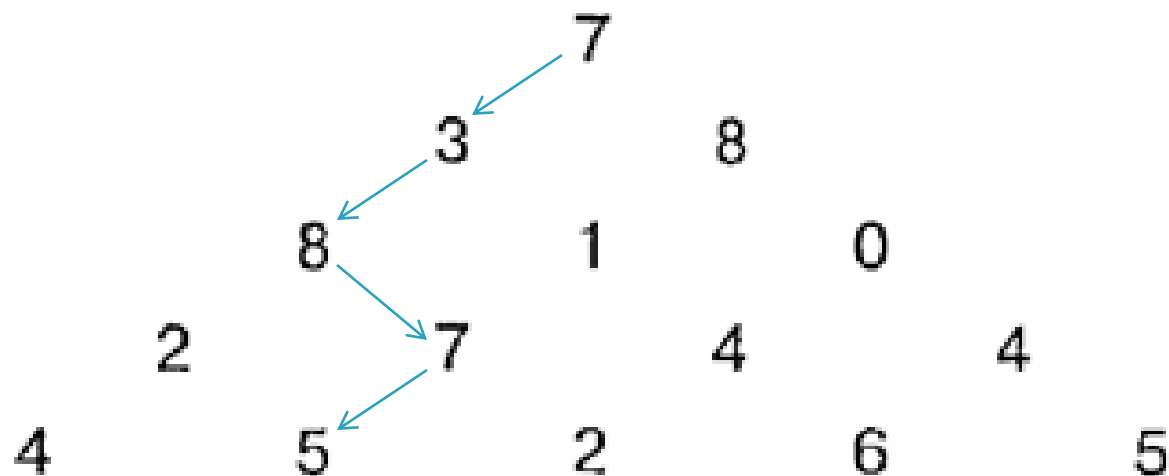
Пример. Треугольник



Пример. Треугольник



Пример. Треугольник



7
 3 8
 8 1 0
 2 7 4 4
 4 5 2 6 5

7	0	0	0	0
3	8	0	0	0
8	1	0	0	0
2	7	4	4	0
4	5	2	6	5

7	0	0	0	0
3	8	0	0	0
8	1	0	0	0
2	7	4	4	0
4	5	2	6	5

R – массив
[1..MaxN,0..MaxN]

0	7				
0	10	15			
0	18	16	15		
0	20	25	20	19	
0	24	30	27	26	24

$R[1,1]=D[1,1]$

For i=2 To N:

For j=1 To i:

$R[i,j]=\max(D[i,j]+R[i-1,j], D[i,j]+R[i-1,j-1])$

Пример. Степень числа

- ▶ Даны два натуральных числа N и k . Требуется определить выражение, которое вычисляет k^N . Разрешается использовать операции умножения и возведения в степень, круглые скобки и переменную с именем k . Умножение считается одной операцией, возведению в степень q соответствует $q-1$ операция
- ▶ Найти минимальное количество операций, необходимое для возведения в степень N . Желательно сделать это для как можно больших значений N

Пример

- ▶ при $N=2$ требуется одна операция – $k*k$
- ▶ при $N=5$ требуются три операции – $(k*k)^2*k$

Логика решения

- ▶ Определим массив Op , его элемент $Op[i]$ предназначен для хранения минимального количества операций при возведении в степень i ($Op[1]=0$)

Логика решения

- ▶ Для вычисления выражения, дающего N -ю степень числа k , арифметические операции применяют в некоторой последовательности, согласно приоритетам и расставленным скобкам. Последняя примененная операция была или умножением или возведением в степень

Логика решения

- ▶ Если это было умножение, тогда сомножителями являются натуральные степени числа k , которые в сумме дают N . Степень каждого из сомножителей меньше N , и ранее вычислено, за какое минимальное число операций ее можно получить:

$$opr1 = \text{Min} \{ \text{по всем } p: 1 \leq p < N, Opr[p] + Opr[N-p] + 1 \}$$

Логика решения

- ▶ Если последней операцией являлось возведение в степень, то
$$opr2 = \text{Min}\{\text{для всех } p (\neq 1) - \text{делителей } N, \\ Op[N \text{ div } p] + p - 1\}$$
- ▶ Результат – $Op[N] = \text{Min}(opr1, opr2)$.

Фрагмент реализации

Const MaxN=1000

Op: массив чисел [1..MaxN]

Процедура Solve:

for i=2 To N :

Op[i] =Op[i-1]+1

for j=2 To i-1 :

Op[i] = Min(Op[i], Op[j]+Op[i-j]+1)

If i Mod j==0:

Op[i] = Min(Op[i], Op[i Div j]+j-1)

Пример: Выражение

- ▶ Дано арифметическое выражение, операндами которого являются целые положительные числа, а разрешенными операциями «+» и «*». Требуется расставить скобки так, чтобы результат вычисления выражения был максимальным

Пример: Выражение

- ▶ Строка s описывает конкретное выражение
- ▶ Подзадача – это решение задачи для подстроки с i -го по j -й операнд. Получив один раз результат для подзадачи, он затем в неизменном виде (без пересчета) используется для решения подзадач большей размерности
- ▶ В массиве W храним результаты решения подзадач, $W[i,j]$ равно максимальному значению выражения, полученного при решении задачи для подстроки $s[i..j]$. Значение $W[i,i]$ равно i -му операнду. Очевидно, что $W[i,j]=0$ при $i > j$

Пример: Выражение

- ▶ Особенность заполнения W заключается в том, что он заполняется по диагоналям, параллельным главной: $j-i=0, j-i=1, j-i=2, \dots, j-i=n-1$. Принцип заполнения описывается следующими рекуррентными соотношениями:

$$W[i, j] = \begin{cases} s[i] & \text{при } i=j, 1 \leq i, j \leq n \\ \text{Max}(W[i, k] \cdot W[k+1, j], W[i, k] + W[k+1, j]) & \text{при } i < j \\ & i \leq k < j \end{cases}$$

Пример: Выражение

- ▶ Подзадачи для строки $s=3\ 1\ 5\ 1\ 2$
- ▶ Подзадачи с номерами 1, затем с номерами 2, затем – 3, 4 и 5

$$P = \begin{pmatrix} 1 & 2 & 3 & 4 & 5 \\ 0 & 1 & 2 & 3 & 4 \\ 0 & 0 & 1 & 2 & 3 \\ 0 & 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix}$$

Пример: Выражение

▶ $s = 3 \ 1 \ 5 \ 1 \ 2$

$$W[i, j] = \begin{cases} s[i] \text{ при } i=j, 1 \leq i, j \leq n \\ \text{Max}(W[i, k] \cdot W[k+1, j], W[i, k] + W[k+1, j]) \text{ при } i < j \\ i \leq k < j \end{cases}$$

$$W = \begin{pmatrix} 3 & 4 & 20 & 24 & 60 \\ 0 & 1 & 6 & 7 & 18 \\ 0 & 0 & 5 & 6 & 15 \\ 0 & 0 & 0 & 1 & 3 \\ 0 & 0 & 0 & 0 & 2 \end{pmatrix}$$

Пример: Выражение

- ▶ Максимальное значение выражения равно $60 = W[1, n]$, однако операции и расстановка скобок не определены. Для того, чтобы их получить, на каждом шаге требуется запоминать (в дополнительной структуре данных) значение k и операцию, на которой достигается максимум
- ▶ Так для рассматриваемого примера выражение имеет вид: $((3 + 1) \bullet 5) \bullet (1 + 2)$ или $(3 + 1) \bullet (5 \bullet (1 + 2))$

Пример.

Алгоритм Нудельмана–Вунша

- ▶ Молекулы ДНК содержат генетическую информацию. Моделью ДНК можно считать длинное слово из четырех букв (А, Г, Ц, Т). Даны два слова (длины M и N), состоящие из букв А, Г, Ц, Т
- ▶ Найти подпоследовательность наибольшей длины, входящую в то и другое слово

- ▶ Слова ГЦАТАГГТЦ и АГЦААТГГТ

Принцип заполнения таблицы

- ▶ закрашены клетки, находящиеся на пересечении строки и столбца с одинаковыми буквами
- ▶ формирование первой строки и первого столбца выполняется до заполнения таблицы и осуществляется так: единицей отмечается первое совпадение, затем эта единица автоматически заносится во все оставшиеся клетки
- ▶ элемент $W[i,j]$ равен наибольшему из чисел $W[i-1,j]$, $W[i,j-1]$, а если клетка $\langle i,j \rangle$ закрашена, то и $W[i-1,j-1]+1$

- Слова ГЦАТАГГТЦ и АГЦААТГГТ

Ц	1	2	3	3	4	5	5	6	7
Т	1	2	2	3	4	5	5	6	7
Г	1	2	2	3	4	4	5	6	6
Г	1	2	2	3	4	4	5	5	5
А	1	1	2	3	4	4	4	4	4
Т	1	1	2	3	3	4	4	4	4
А	1	1	2	3	3	3	3	3	3
Ц	0	1	2	2	2	2	2	2	2
Г	0	1	1	1	1	1	1	1	1
	А	Г	Ц	А	А	Т	Г	Г	Т

Ответ

- ▶ Подпоследовательность формируется при обратном просмотре заполненной таблицы от закрашенной клетки, помеченной максимальным значением
- ▶ Путь – это клетки с метками, отличающимися на единицу, буквы выписываются из закрашенных клеток
- ▶ Для нашего примера две подпоследовательности: ГЦААГГТ и ГЦАТГГТ

Фрагмент основной логики

For i=1 To Len(S1) :

For j=1 To Len(S2):

$A[i, j] = \text{Max}(A[i-1, j], A[i, j-1])$

If S1[i]==S2[j]:

$A[i, j] = \text{Max} (A[i, j], A[i-1, j-1]+1)$

ВыводОтвета(A [Len(S1), Len(S2)])

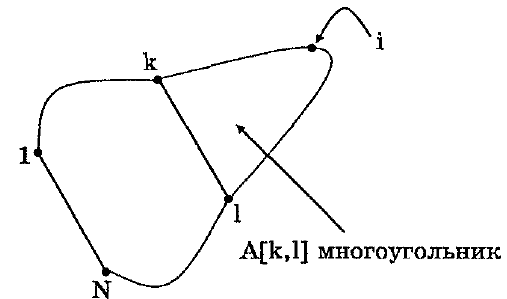
Пример.

Разбиение выпуклого N -угольника

- ▶ Дан выпуклый N -угольник, заданный координатами своих вершин в порядке обхода. Он разрезается $N-3$ диагоналями на $N-2$ треугольника. Стоимость разрезания определяется суммой длин всех использованных диагоналей
- ▶ Найти разрез минимальной стоимости

Идея решения

- Обозначим через $S[k, l]$ стоимость разрезания многоугольника $A[k, l]$ диагоналями на треугольники. При $l = k + 1$ или $k + 2$ $S[k, l] = 0$, следовательно, $l > k + 2$
- Вершина с номером i (i изменяется от $k + 1$ до $l - 1$) определяет какое-то разрезание многоугольника $A[k, l]$. Стоимость разрезания: $S[k, l] = \min\{\text{длина диагонали } \langle k, i \rangle + \text{длина диагонали } \langle i, l \rangle + S[k, i] + S[i, l]\}$. При этом следует учитывать, что при $i = k + 1$ и $i = l - 1$ диагональ $\langle k, i \rangle$ является стороной многоугольника и ее длина считается равной нулю



Пример. Задача о камнях

- ▶ Из камней весом $p_1, p_2, \dots, p_N$ набрать вес W или, если это невозможно, максимально близкий к W снизу вес

Пример

- ▶ $N=5$
- ▶ $p_1=5$
- ▶ $p_2=7$
- ▶ $p_3=9$
- ▶ $p_4=11$
- ▶ $p_5=13$
- ▶ $W=19$

Идея решения

- Сформируем матрицу $A [1..N, 0..W]$, в которой номер строки соответствует номеру камня, а номер столбца – набираемому весу. В нулевой столбец запишем нули, в первую строку (по столбцам) до веса первого камня нули, а затем вес первого камня (пытаемся набрать требуемый вес одним камнем). Во второй строке фиксируем результат набора веса с помощью двух камней и т. д.

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
1	0	0	0	0	0	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5
2	0	0	0	0	0	5	5	7	7	7	7	7	12	12	12	12	12	12	12	12
3	0	0	0	0	0	5	5	7	7	9	9	9	12	12	14	14	16	16	16	16
4	0	0	0	0	0	5	5	7	7	9	9	11	12	12	14	14	16	16	18	18
5	0	0	0	0	0	5	5	7	7	9	9	11	12	13	14	14	16	16	18	18

- ▶ Вес 19 набрать нельзя, ближайший снизу вес 18
- ▶ Для заполнения текущей строки достаточно знать только предыдущую строку (результат решения задачи меньшей на единицу размерности), а в элементе массива $A[i,j]$ записывается решение задачи о камнях при $N=i$ и $W=j$

Фрагмент реализации

Procedure Solve:

For i=2 To N:

For j=1 To W:

If $j - P[i] \geq 0$:

$A[i, j] = \text{Max} (A[i-1, j], A[i-1, j-P[i]] + P[i])$

Else $A[i, j] = A[i-1, j]$

Вывод результата

Procedure Way(i, j):

 If NOT (i==1) And (A[i, j]==0):

 If i==1:

 Way (i, j-P[i])

 Вывод(P[i], ' ')

 Else If A[i, j]<>A[i-1, j]:

 Way(i, j-P[i])

 Вывод(P[i], ' ')

 Else Way(i-1, j)

Пример. Задача о рюкзаке

- ▶ В рюкзак загружаются предметы N различных типов (количество предметов каждого типа не ограничено). Вес рюкзака должен быть меньше или равен W . Каждый предмет типа i имеет вес w_i и стоимость v_i ($i=1, 2, \dots, N$)
- ▶ Требуется определить максимальную стоимость груза, вес которого не превышает W

Решение

- ▶ Рассмотрим случай, когда вес рюкзака в точности равен W . Обозначим количество предметов типа i через k_i тогда требуется максимизировать $v_1 * k_1 + v_2 * k_2 + \dots + v_N * k_N$ при ограничениях:
 - $w_1 * k_1 + w_2 * k_2 + \dots + w_N * k_N = W$,
где k_i – целые ($0 \leq k_i \leq [W / w_i]$)

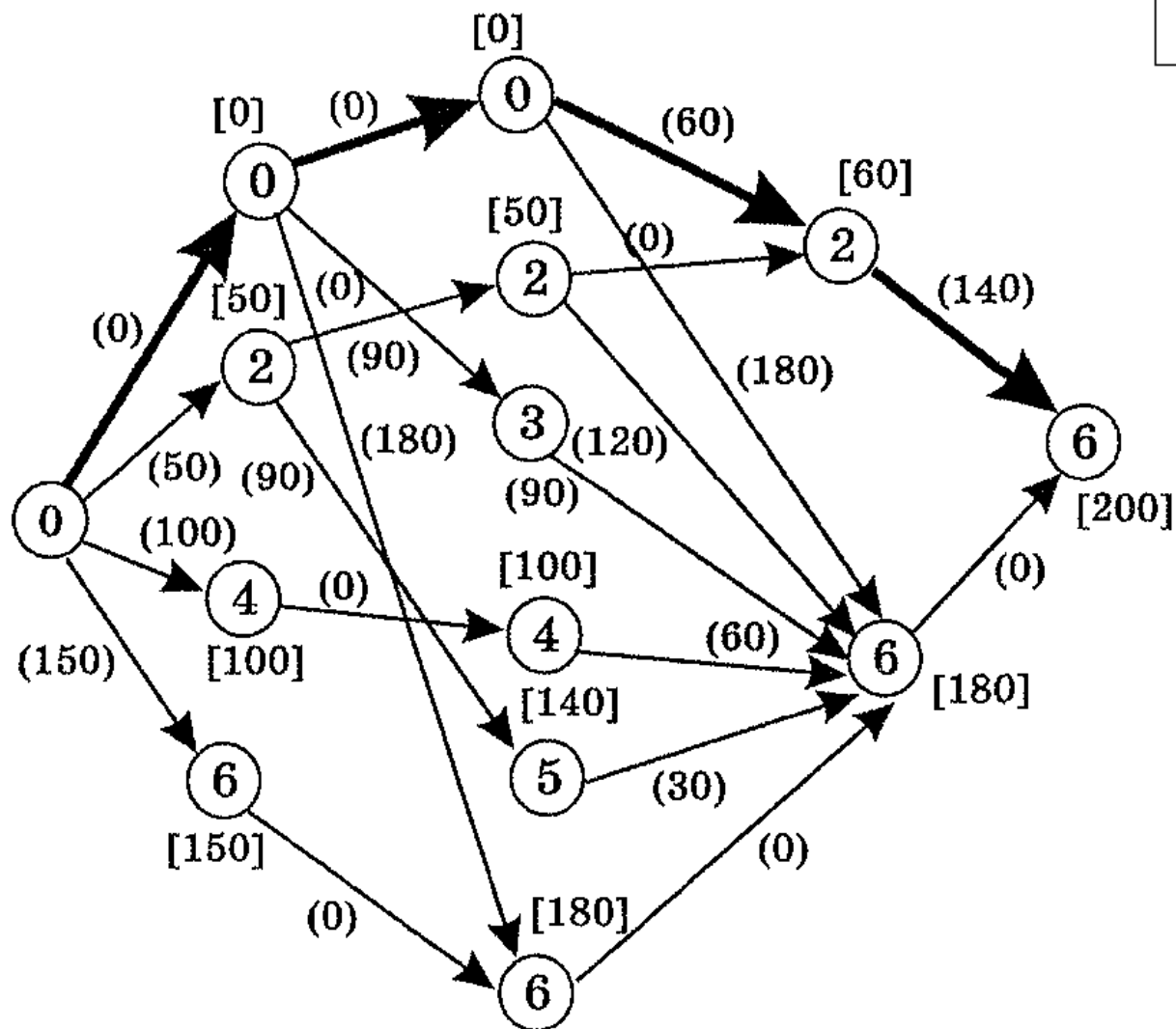
Решение

- ▶ Шаг i ставится в соответствие типу предмета $i=1,2,\dots,N$
- ▶ Состояние y_i на шаге i выражает суммарный вес предметов, решение о загрузке которых принято на шагах $0,1,\dots,i$. При этом $y_n=W$, $y_i=0,1,\dots,W$ при $i=1,2,\dots,N-1$
- ▶ *Варианты решения k_i на шаге i описываются количеством предметов типа i , $0 \leq k_i \leq [W/w_i]$*

Пример

- W=6, дано четыре предмета:

i	w_i	v_i
1	2	50
2	3	90
3	1	30
4	4	140



- ▶ в кружочках выделены только достижимые состояния (суммарные веса для каждого шага) на каждом шаге
- ▶ в круглых скобках – стоимости соответствующих выборов
- ▶ в квадратных скобках – максимальная стоимость данного заполнения рюкзака
- ▶ жирными линиями выделен способ наилучшей загрузки рюкзака

I	W_i	V_i
1	2	50
2	3	90
3	1	30
4	4	140

